

Canvas LMS REST API: Reference Summary

Source: <https://canvas.instructure.com/doc/api/> Fetched: 2026-04-29

1. Authentication

API Tokens (Personal Access Tokens)

- Generated in Canvas profile under “Approved Integrations.” Intended for personal scripting/testing only.
- Pass via **Authorization: Bearer <TOKEN>** header (strongly preferred – avoids token leakage in URLs/logs).
- Also accepted as query string **?access_token=** or POST body, but header is strongly preferred.

OAuth2 - Authorization Code Flow

- Standard flow for user-facing web/mobile applications.
- Redirect user to Canvas, receive authorization code, exchange for access token + refresh token.
- Access tokens expire after one hour. Refresh tokens do not change.

OAuth2 - Client Credentials (LTI Advantage)

- Server-to-server using RSA256-signed JWTs.
- Used for Names and Role Provisioning Services and Assignment/Grade Services.

Security Notes

- HTTPS required on all API calls.
 - Never embed tokens in web pages or expose via URLs.
 - Each OAuth token has its own independent rate-limit quota.
-

2. Request/Response Conventions

Data Format

- All responses: JSON.
- POST/PUT requests: **application/x-www-form-urlencoded** or **application/json**.
- Timestamps: ISO 8601 UTC (e.g., 2025-04-29T14:00:00Z).
- Integer IDs are 64-bit. Send **Accept: application/json+canvas-string-ids** header to receive IDs as strings (important for JavaScript to avoid precision loss).
- Booleans accept: **true/false, t/f, yes/no, y/n, on/off, 1/0**.

Pagination

- Default 10 items/page; adjust with **?per_page=N** (no officially documented maximum, but instances may cap it).
- Navigation via **Link** response header with **rel** values: **current, next, prev, first, last**.
- The **rel="last"** link may be omitted when calculating total count is expensive.
- Links are opaque absolute URLs – follow them, do not construct your own.

Rate Limiting

- Dynamic cost-based system. Each request deducts a floating-point “cost” from a per-token quota that replenishes over time.
- Response headers: **X-Request-Cost, X-Rate-Limit-Remaining**. Exceeding quota returns HTTP 429.
- Sequential requests rarely hit limits. Parallel requests incur a pre-flight penalty.
- Each user’s token has its own independent quota.

Error Codes

- 204: success with no response body.
- 409: conflict (e.g., creating a quiz submission that already exists).
- 422: unprocessable (e.g., canceling a completed report).
- 429: rate limit exceeded.

SIS IDs and Object IDs

- Most endpoints accept Canvas IDs, SIS IDs (prefix **sis_user_id:**), or integration IDs.
 - The special string **self** substitutes for the current authenticated user’s ID.
-

3. Major Resource Categories

Courses

Base path: **/api/v1/courses**

Key endpoints: - **GET /v1/courses** – current user’s courses - **GET /v1/accounts/{account_id}/courses** – all courses in an account (admin) - **GET /v1/courses/{id}** – single course - **PUT /v1/courses/{id}** – update (publish, conclude, settings) - **DELETE /v1/courses/{id}** – delete or conclude

The **include[]** parameter unlocks substantial additional data: **syllabus_body, term, account, sections, total_scores, current_grading_period_scores, teachers, enrollments, course_image, banner_image, grading_periods**.

Course lifecycle states: **unpublished, available, completed, deleted**. Course events (via PUT action): **claim** (unpublish), **offer** (publish), **conclude, delete, undelete**.

Users within a course: **GET /v1/courses/{course_id}/users** with **enrollment_type[]** filter (student, teacher, ta, observer, designer). The **/students** shortcut is deprecated.

Archival tip: fetch with **include[]=syllabus_body,term,sections,teachers** to capture full course meta-data in one call.

Users

Base path: **/api/v1/users**

Key endpoints: - **GET /v1/accounts/{account_id}/users** – list (search by name, email, SIS ID, login) - **GET /v1/users/{id}** – single user; use **self** for current user - **GET /v1/users/{user_id}/profile** – profile including LTI user ID, calendar URL - **GET /v1/users/{user_id}/missing_submissions** – past-due with no submission

search_term minimum is 3 characters; matches canonical ID, SIS ID, login, name, and email simultaneously.

SIS fields (**sis_user_id, sis_login_id**) are only visible to users with appropriate permissions.

Enrollments

Base path: **/api/v1/courses/{course_id}/enrollments**

Types: **StudentEnrollment, TeacherEnrollment, TaEnrollment, ObserverEnrollment, DesignerEnrollment**, plus custom **role_id**-based roles.

States: **active, invited, creation_pending, deleted, rejected, completed, inactive**. Synthetic states for user-scoped queries: **current_and_invited, current_and_future, current_future_and_restricted, current_and_concluded**.

Key fields: **last_activity_at, last_attended_at, total_activity_time**, grade data (**current_score, final_score, current_grade, final_grade**), SIS identifiers.

Grades are scoped to the current grading period when returned on enrollment objects. Unposted grade variants available to privileged users.

Note: Only root-level admins can enumerate other users' enrollments across accounts.

Assignments

Base path: `/api/v1/courses/{course_id}/assignments`

Submission types (array field, can be mixed): `online_upload`, `online_text_entry`, `online_url`, `media_recording`, `student_annotation`, `online_quiz`, `none`, `on_paper`, `discussion_topic`, `external_tool`.

Grading types: `pass_fail`, `percent`, `letter_grade`, `gpa_scale`, `points`, `not_graded`.

Key fields: `needs_grading_count`, `workflow_state` (published/unpublished), `group_category_id`, `peer_reviews`, `moderated_grading`, `score_statistics` (min/max/mean, via `include`).

Assignment overrides allow per-student, per-group, or per-section due/lock/unlock dates. Override precedence: student IDs > group > section.

Bulk date update (PUT `/v1/courses/{course_id}/assignments/bulk_update`) is asynchronous – returns a Progress object to poll.

Submissions

Base path: `/api/v1/courses/{course_id}/assignments/{assignment_id}/submissions`

Submission types: `online_text_entry`, `online_url`, `online_upload`, `media_recording`, `basic_lti_launch`, `student_annotation`.

Key fields: - `workflow_state`: `submitted`, `unsubmitted`, `graded`, `pending_review` - `submitted_at`, `score`, `grade` - `grader_id`: positive = human grader, negative = automated process - `grade_matches_current_submission` false when resubmitted without regrading - `late` (boolean), `attempt`

Grading via PUT: `submission[posted_grade]` (points, percent, letter, pass/fail), `submission[excuse]`, `submission[late_policy_status]` (“late”, “missing”, “extended”, “none”), `rubric_assessment`.

Archival note: for quiz answer data, do NOT use the Quiz Submissions API – it does not return student answers. Use `include[]=submission_history` on the Assignment Submissions API to get `submission_data` containing actual answers.

Quizzes (Classic Quizzes)

Base path: `/api/v1/courses/{course_id}/quizzes`

Quiz types: `practice_quiz`, `assignment`, `graded_survey`, `survey`.

Key behavioral fields: `shuffle_answers`, `time_limit`, `allowed_attempts`, `scoring_policy`, `hide_results` (always or `until_after_last_attempt`), `show_correct_answers`, `show_correct_answers_last_attempt`, `one_question_at_a_time`, `cant_go_back`, `access_code`, `ip_filter`.

Important: cannot unpublish a quiz once submissions exist – the API returns an error. The `unpublishable` field on the quiz object indicates this constraint.

The Quiz Questions endpoint (`/courses/{id}/quizzes/{id}/questions`) returns 403 on UTK Canvas even for instructors. Use Student Analysis CSV reports or the Quiz Submission Questions API instead.

Classic Quizzes and New Quizzes are entirely separate systems with different APIs. This codebase targets Classic Quizzes only.

Quiz Submissions

Base path: `/api/v1/courses/{course_id}/quizzes/{quiz_id}/submissions`

Key fields: - `id`, `quiz_id`, `user_id` - `submission_id`: links to the corresponding Assignment Submission record - `started_at`, `finished_at`, `end_at` - `time_spent`: wall-clock seconds from quiz open to submit -

`extra_time`: minutes of extra time granted - `score`, `kept_score`, `fudge_points` - `attempt` - `workflow_state`: `untaken`, `pending_review`, `complete`, `settings_only`, `preview`

Teacher preview attempts do not count toward student records.

Critical: `submission_data` (actual student answers) is NOT returned by this API. Use the Assignment Submissions API with `include[]=submission_history`.

Quiz Reports

Base path: `/api/v1/courses/{course_id}/quizzes/{quiz_id}/reports`

Types: - `student_analysis`: per-student summary CSV - `item_analysis`: per-question performance CSV

Async 4-step workflow: 1. POST to create – returns immediately with a progress URL. 2. Poll `GET /api/v1/progress/{id}` until `workflow_state == "completed"`. 3. GET the report object to retrieve the file attachment URL. 4. Download the CSV – the URL contains a verifier token; do NOT send an Authorization header on the final download request.

Key parameter: `includes_all_versions` (true = all attempts, false = most recent only).

Reports can only be aborted if still in `queued` state. A completed report must be explicitly deleted before regenerating. Reports go stale if new submissions arrive after generation.

Surveys are not reportable (`generatable: false`).

Quiz Submission Events

Base path: `/api/v1/courses/{course_id}/quizzes/{quiz_id}/submissions/{id}/events`

Captures behavioral events during quiz-taking for academic integrity analysis.

Key fields: - `created_at`: server-recorded timestamp (not client timestamp) - `event_type`: e.g., `question_answered`, `page_blurred`, `page_focused` - `event_data`: flexible object, contents vary by event type

Quirks: - `quiz_question_ids` in event data are returned as strings despite being numeric IDs. - `created_at` is server time; `event_data` may contain a `client_timestamp` field but it is not formally specified in the API.

Events are submitted in batches (array). Retrieval supports `attempt` parameter (defaults to latest).

Discussion Topics

Base path: `/api/v1/courses/{course_id}/discussion_topics`

Discussion types: `side_comment`, `threaded` (fully nested), `not_threaded` (single nesting level).

Best archival endpoint: `GET /v1/courses/{course_id}/discussion_topics/{topic_id}/view` Returns a single non-paginated response with the complete cached discussion: all participants (with IDs and display names), all entries with full message bodies, and threaded replies.

Add `?include_new_entries=1` for eventual consistency (captures entries not yet in the cached view; returned as a flat list in ascending order).

Entry deletion blanks `user_id` and `message` but preserves the record – deleted entries persist with cleared content.

Parallel endpoints exist for groups at `/v1/groups/{group_id}/discussion_topics`.

Files

Base path: `/api/v1/files/{id}` and `/api/v1/courses/{id}/files`

Key fields: `url` (download URL with verifier token), `content-type`, `size`, `created_at`, `updated_at`, `lock_at`, `unlock_at`, `hidden`, `locked`.

Download pattern: file `url` field includes a verifier token in the query string. No Authorization header is needed on the actual download request – the verifier serves as the credential.

Folder hierarchy: GET `/v1/folders/by_path/{full_path}` for path-based lookup, or GET `/v1/courses/{id}/folders` for a flat list of all folders.

File listing supports filtering by `content_types[]`, `search_term`, and sorting by name, size, `created_at`, `updated_at`, `content_type`, or user.

Storage quota: GET `/v1/courses/{id}/files/quota` for planning bulk downloads.

Archival pattern: enumerate folders, enumerate files per folder, download using verifier URLs (no auth header needed on download).

Groups

Base path: `/api/v1/courses/{course_id}/groups`

Group types: course groups (nested in a course), account groups, community groups.

Join levels: `parent_context_auto_join`, `parent_context_request`, `invitation_only`.

Membership workflow states: `accepted`, `invited`, `requested`.

`non_collaborative` flag: groups used for permission-based tagging rather than student collaboration.

Warning: deleting a group removes all memberships – there is no soft-delete or archival state.

Gradebook History

Base path: `/api/v1/courses/{course_id}/gradebook_history`

Key endpoints: - GET `/gradebook_history/days` – map of dates to grader/assignment activity - GET `/gradebook_history/{date}` – graders and assignments for a specific date - GET `/gradebook_history/{date}/graders` – versioned submission list - GET `/gradebook_history/feed` – paginated, uncollated feed of all Submission-Version records (filterable by assignment/user, sortable by date)

SubmissionVersion fields: `current/new/previous` grades, grader identity, timestamps, workflow state, score.

Note: in the feed endpoint, version objects exclude `new_grade` and `previous_grade` prefixes – only current grade info is shown.

Rubrics

Base path: `/api/v1/courses/{course_id}/rubrics`

Structure: `RubricCriteria` array, each with a `ratings` array of `RubricRating` objects. `criterion_use_range` enables point ranges rather than discrete levels.

Association to assignments via `RubricAssociation`: `use_for_grading`, `purpose` (“grading” or “bookmark”), `hide_score_total`.

`RubricAssessment` captures individual grading events: `assessment_type` (grading, peer_review, provisional_grade), `assessor_id`, `artifact_id` (submission reference).

Warning: deleting a rubric cascades to remove all `RubricAssociation` records, potentially losing historical grading metadata.

CSV import/export available: POST `/rubrics/upload` with template from GET `/rubrics/upload_template`.

Content Exports (Bulk Course Archive)

Base path: `/api/v1/courses/{course_id}/content_exports`

Export types: - `common_cartridge`: full course in IMSCC format (IMS Common Cartridge) - `qti`: quizzes in QTI format - `zip`: file storage only

Async workflow: 1. POST to initiate – returns immediately with `progress_url`. 2. Poll GET `/api/v1/progress/{id}` until `workflow_state` == “exported”. 3. GET the export object to retrieve `attachment.url`. 4. Download.

`select` parameter enables partial exports: specific files, folders, pages, quizzes, assignments, announcements, calendar events, discussion topics, modules, module items, rubrics.

4. Data Type Enumerations

Concept	Values
Quiz types	<code>practice_quiz</code> , <code>assignment</code> , <code>graded_survey</code> , <code>survey</code>
Enrollment types	<code>StudentEnrollment</code> , <code>TeacherEnrollment</code> , <code>TaEnrollment</code> , <code>ObserverEnrollment</code> , <code>DesignerEnrollment</code>
Enrollment states	<code>active</code> , <code>invited</code> , <code>creation_pending</code> , <code>deleted</code> , <code>rejected</code> , <code>completed</code> , <code>inactive</code>
Submission types	<code>online_upload</code> , <code>online_text_entry</code> , <code>online_url</code> , <code>media_recording</code> , <code>student_annotation</code> , <code>online_quiz</code> , <code>none</code> , <code>on_paper</code> , <code>discussion_topic</code> , <code>external_tool</code> , <code>basic_lti_launch</code>
Grading types	<code>pass_fail</code> , <code>percent</code> , <code>letter_grade</code> , <code>gpa_scale</code> , <code>points</code> , <code>not_graded</code>
Submission workflow	<code>submitted</code> , <code>unsubmitted</code> , <code>graded</code> , <code>pending_review</code>
Quiz submission workflow	<code>untaken</code> , <code>pending_review</code> , <code>complete</code> , <code>settings_only</code> , <code>preview</code>
Discussion types	<code>side_comment</code> , <code>threaded</code> , <code>not_threaded</code>
Group join levels	<code>parent_context_auto_join</code> , <code>parent_context_request</code> , <code>invitation_only</code>
Rubric assessment types	<code>grading</code> , <code>peer_review</code> , <code>provisional_grade</code>
Export types	<code>common_cartridge</code> , <code>qti</code> , <code>zip</code>
Late policy status	<code>late</code> , <code>missing</code> , <code>extended</code> , <code>none</code>

5. Archival Patterns

Goal	Pattern
Quiz student answers	Assignment Submissions API with <code>include[]=submission_history</code> (provides <code>submission_data</code>)
Quiz question metadata	Quiz Submission Questions API (<code>quiz_questions</code> key) – NOT the Questions endpoint (403 on UTK)
Quiz statistical CSVs	Async: POST report, poll progress, GET file URL, download without auth header
Behavioral events	Quiz Submission Events API (server timestamps; <code>quiz_question_ids</code> are strings)
Discussion full archive	Single <code>/view</code> call per topic (non-paginated, includes all participants + replies)
File downloads	Enumerate via <code>/files</code> , download via <code>url</code> field (verifier token, no auth header needed)

Goal	Pattern
Full course backup	POST /content_exports with export_type=common_cartridge; async poll + download
Grade change history	GET /gradebook_history/feed with order=asc
Reduce round trips	Use include[] parameters aggressively; per_page=100 for pagination
Async operations	All return progress_url; poll until workflow_state == "completed"

6. Key Quirks and Caveats

- **Quiz Questions 403 on UTK:** GET /courses/{id}/quizzes/{id}/questions returns 403 even for instructors on UTK Canvas. Use Student Analysis CSV or Quiz Submission Questions API instead.
- **Quiz Reports download:** Final CSV download URL contains a verifier token; do NOT send an Authorization header, or the download will fail.
- **New vs Classic Quizzes:** Completely separate API surfaces. This codebase targets Classic Quizzes only; New Quizzes use LTI-based APIs.
- **Cannot unpublish quizzes with submissions:** Canvas enforces this at the API level. The unpublishable field signals this state. Never delete a published quiz to replace it.
- **64-bit integer IDs:** Send Accept: application/json+canvas-string-ids if consuming IDs in JavaScript to prevent precision loss.
- **Deletion behavior differs by resource:** Rubric deletion cascades to associations (data loss risk). Group deletion removes all memberships. Discussion entry deletion blanks content but preserves the record. File deletion is permanent unless replaced.
- **Quiz submission events quiz_question_ids:** Returned as strings despite being numeric – parse accordingly.
- **submission_data location:** Not in Quiz Submissions API. Found in Assignment Submissions API via include[]=submission_history.